

Certified Quantum Randomness

Protocol, Mathematics, and Status — a consolidated reference

RCS-VRF Subnet — working notes

May 26, 2026

What this document is. A single ordered reference pulling together the intuition *and* the mathematics behind the RCS-VRF certified-randomness protocol: why it exists, what it must achieve, how the XEB verification works (with a fully worked numerical example), the certified min-entropy bound (anchored to Liu *et al.*, Nature **640**:343, 2025; arXiv:2503.20498), and an honest accounting of what is proven versus still open. Plain-English boxes summarise each technical section.

Contents

Part I — The big picture	2
1 What the protocol is for	2
2 The four required properties	2
3 Who is the “attacker,” and why we model one	3
4 Proving “quantum” by ruling out “classical”	3
5 The timing asymmetry: “a clock, not a computer”	3
Part II — How verification works	4
6 Two roles: prediction vs. measurement	4
6.1 Why sampling is easy but prediction is hard	4
7 How a circuit is simulated (Step 0 in detail)	4
8 The cross-entropy benchmark F_{XEB}	5
8.1 A fully worked example ($n = 2$)	5
9 The mathematics of the reference cases	6
9.1 The full distributions (used by the security proof)	7
Part III — The certified entropy	7
10 Why min-entropy, and extraction	7
11 The heuristic we started with — and why it was wrong	7

12 The honest-server bound (our near-term scope)	8
12.1 Per-round entropy: the exact collision entropy	8
12.2 Accumulation: i.i.d., no EAT, no η	8
13 The certified bound (Liu <i>et al.</i>)	8
14 $Q_{\min}$: the real remaining object	9
15 Aaronson–Hung vs. Liu: which bound we use	9
Part IV — Status and honest accounting	10
16 What is verified, and what remains	10
17 Honest caveats that did not change	10
18 Why security is the whole point	10
19 Summary	11

Part I — The big picture

1 What the protocol is for

The goal is to produce random bits that come with a *proof* they are genuinely unpredictable and genuinely quantum in origin — verifiable by anyone with a laptop, without trusting the hardware provider. A client generates pseudo-random “challenge” circuits from a small seed, sends them to an untrusted quantum server, and asks for one measured bitstring per circuit. The server’s results are then checked, and if they pass, distilled into certified-uniform randomness.

In plain English

We sell “randomness you don’t have to trust us for.” Ordinary random-number generators say “trust me, it’s random.” This protocol instead hands you a number *and* a mathematical certificate that no classical computer could have produced it on demand — so a casino, lottery, or blockchain can verify fairness without trusting the operator.

2 The four required properties

A batch of output must satisfy four properties before it can be sold as certified quantum randomness:

1. **Quantum in origin** — the bits really came from quantum hardware, not a classical RNG. (Established by the XEB score plus the timing deadline.)
2. **Genuinely random** — truly unpredictable and uniform, not merely “quantum-flavoured.” (Min-entropy bound + randomness extraction.)
3. **Classically verifiable** — anyone can check properties 1–2 with only a laptop and the recorded log. (Recompute the score, replay the audit log.)

4. **On-demand** — fresh randomness whenever requested, quickly. (The unpredictable seed drops, the answer returns inside a deadline.)

In plain English

Where did it come from (quantum), is it actually random (provably yes), can outsiders check (yes, classically), can you get it when you ask (yes, fast). Properties 1 and 4 are about the *source*; properties 2 and 3 are about the *guarantee*. They prop each other up: on-demand is *how* we get quantum-in-origin (the deadline rules out faking), and verifiability is *how* anyone trusts the rest.

3 Who is the “attacker,” and why we model one

The attacker is not an outside hacker — it is a *miner*: a participant paid to run the quantum sampling who would rather collect the reward without doing the expensive quantum work (by faking samples classically), or who wants to *bias* the output to win a bet. We assume the worst-case: the most capable, most motivated such participant.

In plain English

Defending against the attacker *is* the product. Strip it out and you’ve built an expensive RNG that says “trust me.” The value is precisely that the output holds up against a participant with money on the line and every reason to cheat. We assume the worst case because security that only works against lazy attackers is luck, not security.

4 Proving “quantum” by ruling out “classical”

We cannot directly inspect qubits. Instead we rule out every *feasible classical* way the samples could have been produced; what remains is genuine quantum sampling. The list of “classical methods to rule out” is identical to the “attacker playbook,” so one argument does both jobs.

The claim is “not *feasibly* classical,” not “not classical in principle.” A classical computer *can* reproduce the samples given enough time; we only prove it cannot do so *here, now, on demand*, within the deadline. The proof is therefore only as strong as our list of classical strategies being complete — which is why the hardness of spoofing XEB is an active research assumption, not a theorem.

5 The timing asymmetry: “a clock, not a computer”

The attacker and the verifier face the *same* hard calculation, but under opposite time pressure. The attacker must produce passing samples *fast and on demand*; the verifier only has to *recognise* them *slowly and offline, after the fact*.

In plain English

The quantum machine answers in seconds (it just measures). A classical faker must simulate the circuit, which takes hours–days, so it misses the deadline. The verifier checks the recorded log later with no clock running, so it can take all the time it needs. Two legs exist purely to police this clock: the VRF seed stops the attacker starting early, the timing check enforces the deadline. The security is the *gap in deadlines*, not a gap in horsepower.

Part II — How verification works (the XEB machinery)

6 Two roles: prediction vs. measurement

The same distribution — the circuit’s ideal output probabilities $p_C(x) = |\langle x|C|0\rangle|^2$ — is used by both parties, in opposite senses.

- **Step 0 (verifier, prediction).** Computes the probabilities by classically simulating the circuit. Knows the whole “map,” but cannot sample cheaply. Deterministic; contains *no* randomness. Exponentially expensive.
- **Step 1 (server, measurement).** Runs the circuit and measures, collapsing the state to one bitstring with probability exactly $p_C(x)$ (Born rule). Effortless for a quantum device, but yields one draw and no knowledge of the odds. This is the *only* step where randomness enters.

In plain English

Step 0 is the verifier drawing a perfect *weather map* — every region’s rain probability, computed exactly but at great cost. Step 1 is the server reporting whether it’s *actually raining right now* — one real observation, instant, but unpredictable even to someone holding the perfect map. The protocol works because the map is expensive to draw (a faker can’t quickly forge one) and the actual weather is genuinely unpredictable (no map pre-determines it). Even the verifier, who knows the entire distribution, cannot predict the outcome of a single measurement — that residual unpredictability is the certified randomness.

6.1 Why sampling is easy but prediction is hard

Sampling (Step 1) is free for a quantum computer — it is what the hardware physically does. Predicting (Step 0) means computing amplitudes, which is exponentially costly. A classical faker is asked to do the easy-for-quantum thing (sample) and the only classical route is to first do the hard-for-classical thing (predict). That asymmetry is the resource the protocol monetises.

7 How a circuit is simulated (Step 0 in detail)

Two clarifications:

- **The gates are not random at simulation time.** The circuit is pseudo-random but generated *deterministically from the seed* (single-qubit $SU(2)$ gates from a seeded PRG; two-qubit

topology fixed by an edge-colouring). Once the seed is fixed the circuit is one concrete, fully-determined object that verifier and server both reconstruct.

- **Simulation computes one exact amplitude, run once — not many shots.** The verifier needs $p_C(x_i)$ only for the specific strings the server returned. For each it computes the single number $\langle x_i | C | 0 \rangle$ *exactly, once*. No statistics, no repetition.

Computing $\langle x | C | 0 \rangle$ for $n = 56$ cannot use the naive 2^n state vector ($2^{56} \approx 7 \times 10^{16}$ amplitudes). Instead one uses **tensor-network contraction**: represent each gate as a small tensor, fix input $|0\rangle$ and output x , and contract the network down to a single scalar without ever materialising the full state. Index slicing breaks the contraction into GPU-sized pieces.

The per-circuit simulation cost is $B \approx 90 \times 10^{18}$ FLOPs — about 100 seconds on the full Frontier supercomputer for *one* amplitude. Crucially, B is both the verifier’s cost (paid offline, no clock) *and* the cost a classical faker would pay to predict the distribution (which it must pay *within the deadline*). That shared cost, under opposite time pressure, is the security argument made quantitative.

8 The cross-entropy benchmark F_{XEB}

The verification statistic is the linear cross-entropy benchmark, over a random test subset V of size m :

$$F_{\text{XEB}} = \frac{N}{m} \sum_{i \in V} p_{C_i}(x_i) - 1, \quad N = 2^n. \quad (1)$$

Up to constants it is the *average ideal-probability of the strings the server returned*, rescaled so that blind guessing scores 0 and perfect quantum sampling scores 1.

In plain English

F_{XEB} asks: “do the strings the server sent land on the circuit’s *likely* outcomes more than chance would?” An honest quantum machine returns high-probability strings (the Born rule biases it that way), so the average is high. A blind guesser scatters everywhere and averages out to chance. The score is built so chance = 0 and perfect = 1.

8.1 A fully worked example ($n = 2$)

Take $N = 2^2 = 4$, uniform baseline $1/N = 0.25$. Suppose the verifier’s simulation (Step 0) gives

$$p_C(00) = 0.50, \quad p_C(01) = 0.30, \quad p_C(10) = 0.15, \quad p_C(11) = 0.05.$$

The server returns $m = 5$ samples (Step 1): 00, 01, 00, 11, 01. Then:

Step	Operation	Result
2	look up each returned string’s p_C	0.50, 0.30, 0.50, 0.05, 0.30
3	sum	$\sum p_C(x_i) = 1.65$
4	average ($\div m$)	$1.65/5 = 0.33$
5	rescale ($\times N$) so chance = 1	$4 \times 0.33 = 1.32$
6	subtract 1 so chance = 0	$F_{\text{XEB}} = 0.32$

Each step has one job: Step 2 connects samples to the circuit, Step 4 averages out per-sample noise, Step 5 sets the unit, Step 6 sets the zero. Sanity checks on the same circuit: blind guessing (all four strings equally) averages $p_C \rightarrow 0.25$, giving $F_{\text{XEB}} = 4(0.25) - 1 = 0$; returning only the top string 00 gives $4(0.50) - 1 = 1$.

In plain English

Strings come in $\rightarrow$ look up each one's ideal probability $\rightarrow$ average them $\rightarrow$ rescale so chance = 1 $\rightarrow$ subtract so chance = 0. The arithmetic is trivial; the expensive part is Step 2's lookups, because computing each p_C means simulating the circuit.

9 The mathematics of the reference cases

Everything follows from the **Porter–Thomas** law for a random circuit's output probabilities,

$$f(p) = N e^{-Np}, \quad \mathbb{E}[p] = \frac{1}{N}. \quad (2)$$

$F_{\text{XEB}} = 0$ (**uniform / spoof**). A string chosen independently of the circuit lands on the plain PT mean:

$$\mathbb{E}_{x \sim U}[p_C(x)] = \int_0^\infty p f(p) dp = \frac{1}{N} \implies F_{\text{XEB}} = \frac{N}{m} m \frac{1}{N} - 1 = 0.$$

$F_{\text{XEB}} = 1$ (**perfect quantum**). Quantum sampling returns x with probability $p_C(x)$, *size-biasing* the density by an extra factor p :

$$\text{PDF}_{x \sim Q}[p] = N f(p) p = N^2 p e^{-Np}, \quad \mathbb{E}_{x \sim Q}[p_C(x)] = \int_0^\infty p (N^2 p e^{-Np}) dp = N^2 \frac{2}{N^3} = \frac{2}{N}.$$

The sampled probability is *twice* the uniform mean — that factor of 2 is the quantum signal:

$$F_{\text{XEB}} = \frac{N}{m} m \frac{2}{N} - 1 = 1.$$

$0 < F_{\text{XEB}} = \phi$ (**noisy real machine**). A device of fidelity ϕ is modelled (depolarising) as $\rho_\phi = \phi |\psi\rangle\langle\psi| + (1 - \phi) I/N$, whose sampling PDF is the convex mixture

$$\text{PDF}_{x \sim Q_\phi}[p] = \phi \text{PDF}_Q[p] + (1 - \phi) \text{PDF}_U[p], \quad \mathbb{E}[p_C(x)] = \phi \frac{2}{N} + (1 - \phi) \frac{1}{N} = \frac{1 + \phi}{N}.$$

Hence the headline relationship

$$\boxed{F_{\text{XEB}} = (1 + \phi) - 1 = \phi} \quad (3)$$

The score *equals the fidelity*: Liu *et al.* had $\phi \approx 0.3$ and measured $F_{\text{XEB}} = 0.32$.

$F_{\text{XEB}} > 1$ (**cherry-picking — why the band is two-sided**). A cheater that computes p_C and returns only the *highest*-probability strings reports the expected maximum, $\mathbb{E}[\max_x p_C(x)] \approx \frac{\ln N}{N} = \frac{n \ln 2}{N}$, giving $F_{\text{XEB}} \approx n \ln 2 - 1 > 1$. No physical noisy device can exceed ~ 1 (noise only pulls down). Hence the acceptance rule is two-sided,

$$\chi_{\text{low}} \leq F_{\text{XEB}} \leq \chi_{\text{high}},$$

the low edge killing uniform spoofer, the high edge killing cherry-pickers.

$F_{\text{XEB}} \approx \phi$ holds only in the *low-noise* regime ($\epsilon n < \ln 3 \approx 1.1$; the experiment sits at $\epsilon n \approx 0.13$). Past that threshold score and fidelity decouple — one can get high F_{XEB} at low fidelity. So “score = fidelity” is a regime-dependent truth, not an identity.

9.1 The full distributions (used by the security proof)

The means above are the centres; the proof needs the CDFs. Because $\sum_i p_{C_i}(x_i)$ is a sum of exponential-type variables, the XEB score follows **Erlang**/regularised incomplete-gamma ($\tilde{\Gamma}$) laws:

$$\Pr[F_{\text{XEB}} \leq \chi] = \begin{cases} \tilde{\Gamma}(m, m(\chi + 1)) & \text{uniform (0 quantum samples),} \\ \tilde{\Gamma}(2m, m(\chi + 1)) & \text{perfect quantum,} \\ \tilde{\Gamma}(m + l, m(\chi + 1)) & \text{mixture of } l \text{ quantum} + (m - l) \text{ uniform.} \end{cases}$$

The mixture form $\tilde{\Gamma}(m + l, \dots)$ is the workhorse of the $Q_{\min}$ calculation (Part III).

Part III — The certified entropy (Leg 5)

10 Why min-entropy, and extraction

Passing XEB shows the samples are quantum, but they are not yet *uniform* — a real device’s bits are biased and partly predictable. The relevant currency is the **smooth min-entropy** $H_{\min}^{\epsilon_s}$, which measures how many truly-uniform bits can be squeezed out. A randomness extractor (here a Toeplitz hash) then compresses the raw bits to that many uniform bits. Garbage-in-garbage-out applies: the extractor is only as sound as the min-entropy number it is handed.

11 The heuristic we started with — and why it was wrong

The first implementation used

$$H_{\min} \approx m \cdot F_{\text{XEB}} \cdot (n - \log_2 n - \log_2 \ln 2),$$

giving ≈ 1659 bits at $n = 8, m = 1000, F_{\text{XEB}} = 0.3$. This was *formulated, not cited*: three plausible factors multiplied. Only one is legitimate (the per-sample Porter–Thomas min-entropy $n - \log_2 n - \log_2 \ln 2$). The other two are wrong:

- $\times F_{\text{XEB}}$ treats the score as a per-sample *multiplier* — but the score is a *gate* (pass/fail), not a dial.
- the per-sample constant is the wrong quantity for accumulation (see below).

In plain English

The heuristic was an educated placeholder so the pipeline could run — a formula with the right variables but invented glue. The danger was never the placeholder; it was forgetting it was one and quoting its number as if certified. The fix below replaces it with a published theorem.

12 The honest-server bound (our near-term scope)

Assumption 1 (Honest server). Every accepted round is sampled faithfully on the QPU. In the paper’s notation this is the special case $Q_{\min} = M$.

12.1 Per-round entropy: the exact collision entropy

Lemma 1 (Porter–Thomas collision entropy; Liu *et al.* SM Eq. III.33).

$$H_2(X_i | C_i) = -\log_2 \left(\int_0^\infty N f(p) p^2 dp \right) = -\log_2 \left(N^2 \frac{2}{N^3} \right) = -\log_2 \frac{2}{N} = n - 1.$$

The per-round quantity is the *exact* 2-Rényi (collision) entropy, $n - 1$ — not a conservative von Neumann floor. (An earlier draft of ours justified $n - 1$ as a “von Neumann floor with 0.39 bits of slack”; the value was right, the reasoning wrong. The von Neumann entropy of Porter–Thomas is $n - (1 - \gamma)/\ln 2 \approx n - 0.610$ and plays no role here.) The collision entropy is what the i.i.d. min-entropy accumulation requires.

12.2 Accumulation: i.i.d., no EAT, no η

Because each quantum round is i.i.d. (a stated adversary restriction), the smooth min-entropy of Q rounds obeys the Tomamichel–Colbeck–Renner i.i.d. bound:

$$H_{\min}^{\varepsilon_s}(X_Q | \tilde{I}_{\text{sn}}) \geq Q(n - 1) - \log_2 \frac{1}{\varepsilon_s}.$$

There is *no* $\sqrt{M}$ second-order term and *no* generalized-EAT constant η to import — a seam we once worried about that simply does not appear in the implemented bound.

13 The certified bound (Liu *et al.*)

Theorem 1 (Liu *et al.* Theorem 1 / SM Theorem 3, Eq. III.30). *Given $\varepsilon_s \in (0, \frac{1}{4})$, the protocol either aborts with probability $> 1 - 4\varepsilon_s$, or*

$$H_{\min}^{\varepsilon_s}(X^M | \tilde{I}_{\text{sn}}) \geq Q_{\min}(n - 1) - \log_2 \frac{1}{\varepsilon_s}$$

Extraction with a quantum-proof strong (Toeplitz) extractor gives (SM Corollary 7):

$$\ell \leq Q_{\min}(n - 1) - 3 \log_2 \frac{1}{\varepsilon_{\text{sou}}} - 2.$$

Verified against the paper. At $n = 56$, $Q_{\min} = 1297$, $\varepsilon_{\text{sou}} = 10^{-6}$, $\varepsilon_s = \varepsilon_{\text{sou}}/4$:

$$H_{\min} \geq 1297 \cdot 55 - \log_2 \frac{1}{2.5 \times 10^{-7}} = 71,335 - 21.9 = \mathbf{71,313} \text{ bits,}$$

matching the paper exactly; and $\ell \leq 71,335 - 3 \log_2 10^6 - 2 = \mathbf{71,273}$ bits, exactly what they extracted. Our code reproduces both.

Our toy honest-server case. With $Q_{\min} = M$ (Assumption 1), $n = 8$, $M = 1000$, $\varepsilon_s = 2^{-33}$:

$$H_{\min} \geq 1000 \cdot 7 - 33 = 6967 \text{ bits} \quad (\text{vs. the heuristic } 1659).$$

The $\approx 4\times$ gap is exactly the heuristic’s two errors (spurious $\times F_{\text{XEB}}$, wrong constant).

14 $Q_{\min}$: the real remaining object

The score χ and the deadline enter *only* through $Q_{\min}$, the forced quantum-round count (Liu SM Lemma 2):

$$\begin{aligned} \Phi_{\mathcal{A}} &= \min\left(M - Q, \frac{c_{\text{eff}} \mathcal{A}(M t_{\text{th}})}{B}\right), & \varepsilon_1 &= \exp\left(-\frac{\delta^2 \Phi_{\mathcal{A}}}{3}\right), \\ L_{\max} &= Q + \Phi_{\mathcal{A}}(1 + \delta), & \varepsilon_2 &= \sum_{l=0}^m \text{Hyp}(l; M, L_{\max}, m) \tilde{\Gamma}^c(m+l, m(\chi+1)), \end{aligned}$$

with $\varepsilon_{\text{adv}} = \min_{\delta}(\varepsilon_1 + \varepsilon_2)$ and $Q_{\min} = \min\{Q : \varepsilon_{\text{adv}} \geq 4\varepsilon_s\}$. Here $\mathcal{A}$ is the adversary’s FLOP rate, B the per-circuit cost, c_{eff} numerical efficiency.

In plain English

There is no “score $\rightarrow$ entropy curve” to fill in: the per-round entropy is a flat $n - 1$, and the score’s whole role is to set $Q_{\min}$ — how many rounds the attacker is forced to run on real hardware. Computing $Q_{\min}$ is the adversary analysis, and it is what makes the certified number positive only at large scale.

$Q_{\min}$ is transcribed, not yet calibrated. Our `compute_q_min` implements the formulas faithfully and is structurally correct (monotone, sensible), but with the paper’s Table-I values it reproduces $Q_{\min}$ only to order of magnitude (~ 1700 vs 1297); the exact conventions for B , c_{eff} , $\mathcal{A}$ and the δ -rule live in the authors’ released code (Zenodo 10.5281/zenodo.12952178). We deliberately did *not* tune parameters to force 1297 — that would be fitting to the answer. Reconciling against their code turns “reproduce 1297” into a permanent regression test.

15 Aaronson–Hung vs. Liu: which bound we use

We use the **Liu *et al.*** bound, which is the *corrected, finite-size, one-sample-per-circuit* descendant of Aaronson–Hung (AH, arXiv:2303.01625). AH is the asymptotic ancestor: it shows passing XEB forces $\Omega(n)$ bits, but with unknown finite-size constants, for many-samples-from-one-circuit, and with a technical flaw (its single-round *min*-entropy bound is impossible — the 99%-honest/1%-zeros counterexample). Liu corrects it (bounding von Neumann/collision entropy), adapts it to one sample per circuit, and replaces the asymptotics with a concrete restricted-adversary analysis that yields actual numbers.

In plain English

Say “based on the Aaronson–Hung framework,” but the precise, defensible statement is “we use Liu *et al.*’s corrected finite-size bound, which adapts Aaronson–Hung to one-sample-per-circuit.” AH alone does not give a number you can extract against at finite scale.

Part IV — Status and honest accounting

16 What is verified, and what remains

Verified / closed	Remaining
Per-round entropy $H_2 = n - 1$ (exact, Lemma 1).	Calibrate $Q_{\min}$ to the authors' code (Mehdi).
Certified bound reproduces 71,313 bits.	Real QPU for Leg 4 (Property 1; Colton).
Toeplitz extraction reproduces 71,273; extractor is quantum-proof (closes the quantum-conditional-LHL question).	Timing check in Leg 5 (Property 4).
No η /EAT term needed (i.i.d. accumulation).	At production scale, $Q_{\min}$ becomes the central security argument.

17 Honest caveats that did not change

- **Toy scale is ≈ 0 adversarially.** At $n = 8$ a laptop simulates every circuit, so $Q_{\min} \rightarrow 0$ and the certified entropy is ≈ 0 — by design. The positive 6967 exists *only* under the honest-server assumption; quote it with that label.
- **The two biggest product gaps are untouched by the math.** Leg 4 is still a simulator (Property 1) and the timing check is still stubbed (Property 4). Those are hardware/engineering problems, not math problems.
- **Verifiability has limits at scale.** The regime where security switches on is roughly the regime where brute-force classical verification dies, which is why the protocol needs the F_{XEB} estimator and spot-checking rather than full verification.

18 Why security is the whole point

“Certified quantum” and “secure” are the *same* claim: both reduce to “no feasible classical machine could have produced this on demand.” So the proof-of-quantumness and the proof-of-security are one proof. Commercially, the only thing worth a premium over a free classical RNG is the certificate; the load-bearing market question (a Colton question) is whether customers will pay for *physical* unpredictability over a cheaper *cryptographic* one (e.g. Chainlink VRF). Likely yes in high-stakes, regulated, or adversarial settings; worth validating with a concrete willingness-to-pay signal before the expensive QPU spend.

19 Summary

The randomness-certification math is now anchored to a published, verified theorem (Liu *et al.*), reproducing their exact numbers, with two of our own earlier errors corrected and one phantom “open seam” (η/EAT) dissolved. The single genuinely-open piece is the $Q_{\min}$ calibration, now reduced to a bounded regression-test task. This is real progress in *rigour and trustworthiness* — not in toy-scale capability, which remains gated on a real QPU (Leg 4) and the timing check (Leg 5). Every later claim now rests on something checkable.

References. M. Liu *et al.*, *Certified randomness using a trapped-ion quantum processor*, Nature **640**:343–348 (2025); arXiv:2503.20498 (Methods + Supplemental Material). S. Aaronson & S.-H. Hung, *Certified randomness from quantum supremacy*, STOC 2023; arXiv:2303.01625. M. Tomamichel, R. Colbeck, R. Renner, IEEE Trans. Inf. Theory **55**, 5840 (2009). F. Dupuis, O. Fawzi, R. Renner, Commun. Math. Phys. **379**, 867 (2020).